# Path Tracer — Assignment 1 Report

Neyl Nasr

May 18th 2026

## 1 Introduction and Setup

This report presents the path tracer I developed over the four lab sessions, starting from the course skeleton code and progressively implementing the main rendering features: ray–sphere intersection, direct lighting, hard shadows, mirror reflection, indirect lighting, anti-aliasing, ray–mesh intersection, and BVH acceleration.

Unless stated otherwise, the images were rendered at $1024 \times 1024$ resolution with 64 samples per pixel and a maximum recursion depth of 5. The scene is a Cornell-box-style setup made from large spheres acting as walls, floor, and ceiling. The point light is located at $(-10, 20, 40)$ with intensity $I = 3 \times 10^7$. Output colors are gamma-corrected with $\gamma = 2.2$. The rendering loop is parallelized with OpenMP over image rows, and timings were measured on an Apple M3 Pro.

The provided project structure included the general C++ framework and helper files for image input/output and OBJ loading. My implementation focused on the ray-object intersection routines, lighting, sampling, mesh intersection, BVH construction and traversal, and the final rendering experiments.

## 2 Ray–Sphere Intersection, Materials, and Shadows

A ray is parameterized as

$$\mathbf{r}(t) = \mathbf{O} + t\mathbf{u},$$

where $\mathbf{O}$ is the origin, $\mathbf{u}$ is the normalized direction, and $t \geq 0$. For a sphere of center $\mathbf{C}$ and radius $R$, I solve

$$\|\mathbf{O} + t\mathbf{u} - \mathbf{C}\|^2 = R^2,$$

which gives

$$t^2 + 2(\mathbf{O} - \mathbf{C}) \cdot \mathbf{u}\, t + \|\mathbf{O} - \mathbf{C}\|^2 - R^2 = 0.$$

The discriminant is

$$\Delta = ((\mathbf{O} - \mathbf{C}) \cdot \mathbf{u})^2 - (\|\mathbf{O} - \mathbf{C}\|^2 - R^2).$$

If $\Delta < 0$, the ray misses the sphere. Otherwise, I keep the smallest positive root. The hit point and normal are

$$\mathbf{P} = \mathbf{O} + t\mathbf{u}, \qquad \mathbf{N} = \frac{\mathbf{P} - \mathbf{C}}{R}.$$

For diffuse surfaces, I use a Lambertian BRDF. The direct lighting contribution from a point light is

$$L_{\text{direct}} = \frac{I}{4\pi\|\mathbf{L} - \mathbf{P}\|^2} \cdot \frac{\rho}{\pi} \cdot \max(0, \mathbf{N} \cdot \hat{\mathbf{l}}),$$

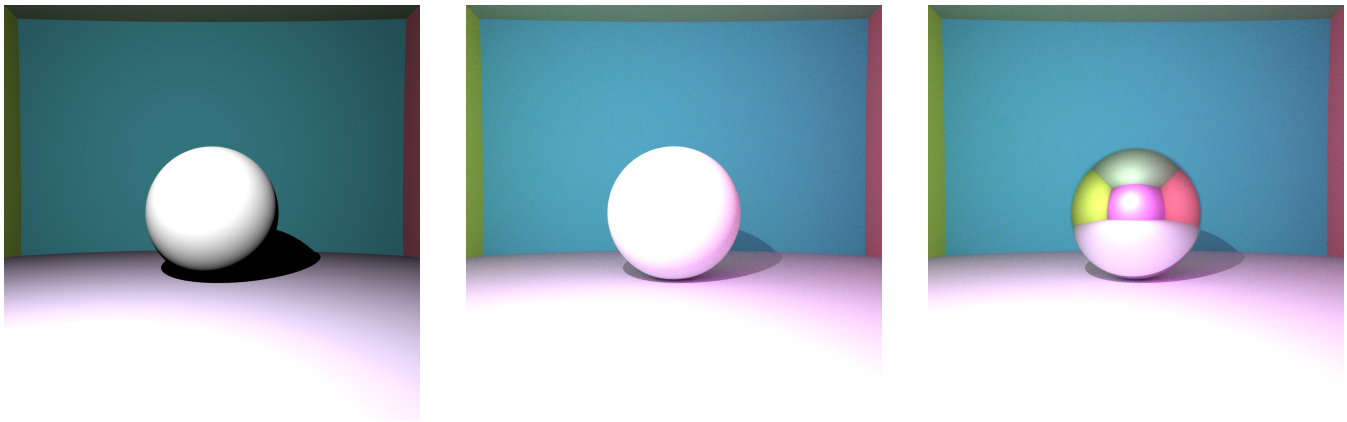
where $\rho$ is the albedo and $\hat{\mathbf{l}} = (\mathbf{L} - \mathbf{P})/\|\mathbf{L} - \mathbf{P}\|$.

For mirror surfaces, the reflected direction is

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{N})\mathbf{N},$$

and `getColor` is called recursively along that reflected ray. Recursion is capped at depth 5.

To add shadows, I send a shadow ray from the hit point toward the light. If another object is intersected before the light, the point receives no direct lighting. To avoid self-intersection, the origin of secondary rays is shifted by $\varepsilon\mathbf{N}$, with $\varepsilon = 10^{-6}$.



(a) Direct lighting. (b) Diffuse sphere. (c) Mirror sphere.

Figure 1: Basic sphere rendering results. Direct lighting creates hard shadows, while the mirror surface reflects the surrounding colored walls.

## 3 Indirect Lighting and Anti-Aliasing

To simulate global illumination, I generate one random secondary ray at each diffuse hit point and recursively estimate the incoming radiance. Directions are sampled over the hemisphere using cosine-weighted sampling. Given $r_1, r_2 \in [0, 1)$, a local sample is

$$x = \cos(2\pi r_1)\sqrt{1 - r_2}, \qquad y = \sin(2\pi r_1)\sqrt{1 - r_2}, \qquad z = \sqrt{r_2}.$$

This direction is transformed into world space using an orthonormal frame $(\mathbf{e}_1, \mathbf{e}_2, \mathbf{N})$:

$$\mathbf{d}_{\text{indirect}} = x\mathbf{e}_1 + y\mathbf{e}_2 + z\mathbf{N}.$$

Because the sampling density is proportional to $\cos\theta$, the cosine term in the Lambertian BRDF cancels with the probability density function. Therefore, the indirect contribution simplifies to

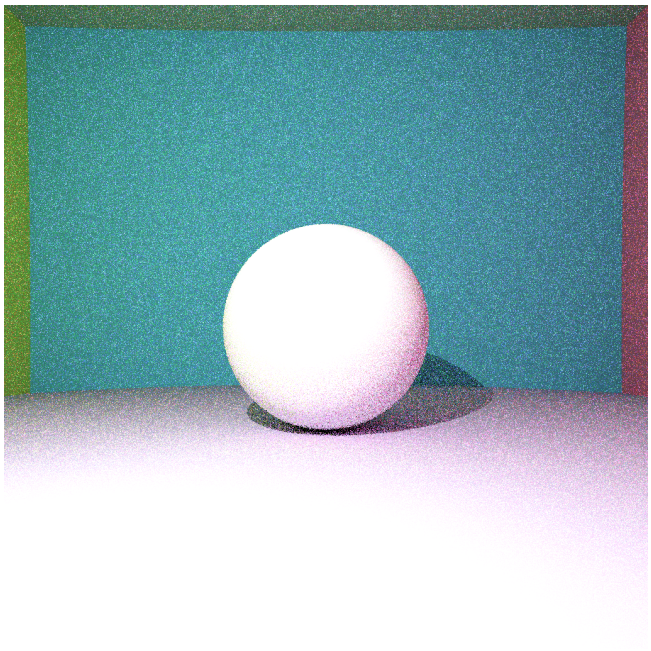
$$L_{\text{indirect}} \approx \rho \cdot L(\text{next hit}).$$

I do not use Russian roulette termination; instead, paths stop after a fixed maximum depth.

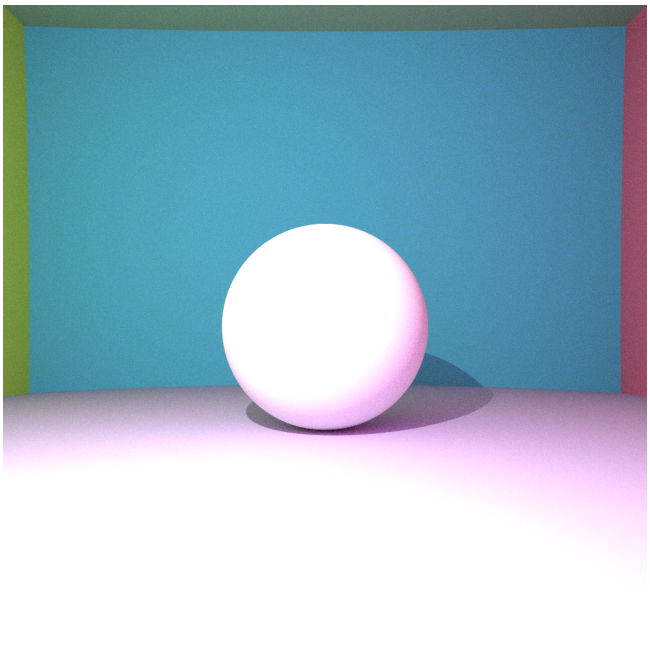
Anti-aliasing is implemented by jittering the primary ray inside each pixel. For each sample, I use a random sub-pixel offset

$$(u - 0.5,\ v - 0.5), \qquad u, v \in [0, 1).$$

The final pixel color is the average over all samples. Since the renderer already uses multiple samples per pixel for Monte Carlo path tracing, anti-aliasing fits naturally into the same loop.



(a) Indirect lighting, no anti-aliasing (1 SPP).



(b) Indirect lighting, anti-aliased (64 SPP).

Figure 2: Anti-aliasing comparison. Both images include indirect lighting. Jittered sampling smooths the jagged edges visible along sphere boundaries in (a).

## 4 Ray–Mesh Intersection

In Lab 3, I added support for triangle meshes. The cat model is loaded from an OBJ file and represented as a list of triangles. The cat mesh contains 3,954 triangles, which makes acceleration necessary for practical render times.

For each triangle with vertices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$, I use the Möller–Trumbore intersection algorithm. Let

$$\mathbf{e}_1 = \mathbf{B} - \mathbf{A}, \qquad \mathbf{e}_2 = \mathbf{C} - \mathbf{A}.$$

The barycentric coordinates and ray parameter are

$$\beta = \frac{\mathbf{e}_2 \cdot ((\mathbf{A} - \mathbf{O}) \times \mathbf{u})}{\mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)}, \qquad \gamma = \frac{-\mathbf{e}_1 \cdot ((\mathbf{A} - \mathbf{O}) \times \mathbf{u})}{\mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)},$$

$$\alpha = 1 - \beta - \gamma, \qquad t = \frac{(\mathbf{A} - \mathbf{O}) \cdot (\mathbf{e}_1 \times \mathbf{e}_2)}{\mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)}.$$

A valid hit satisfies

$$\alpha \geq 0, \qquad \beta \geq 0, \qquad \gamma \geq 0, \qquad t > 0.$$

When several triangles are hit, I keep the smallest positive $t$. This closest-hit rule is important because a ray may intersect several triangles along its path, but only the first visible surface should contribute to the pixel color.

## 5 BVH Acceleration and Results

Testing every ray against every triangle is too slow for larger meshes. I therefore implemented a Bounding Volume Hierarchy. Each BVH node stores an axis-aligned bounding box around a subset of triangles. The tree is built recursively: first, the bounding box of the current triangle range is computed; if the range contains fewer than 5 triangles, the node becomes a leaf. Otherwise, the longest axis of the box is selected, and triangles are split according to the midpoint of their barycenters along that axis. If the split is degenerate, I use a median split instead.

During traversal, the ray is first tested against the bounding box of the current node using the slab method. If the ray misses the box, the whole subtree is skipped. If the node is a leaf, I test only the few triangles stored in that leaf using Möller–Trumbore. This reduces the number of triangle intersection tests significantly, especially for rays that miss the mesh or intersect only a small region of it.

A naive mesh intersection would require testing all 3,954 triangles for every ray. This becomes especially expensive once indirect bounces and multiple samples per pixel are enabled, because each primary ray may generate several secondary rays. The BVH changes this behavior by rejecting large groups of triangles at once through bounding-box tests. This is useful not only for primary camera rays, but also for shadow rays and indirect rays, many of which miss the mesh entirely.

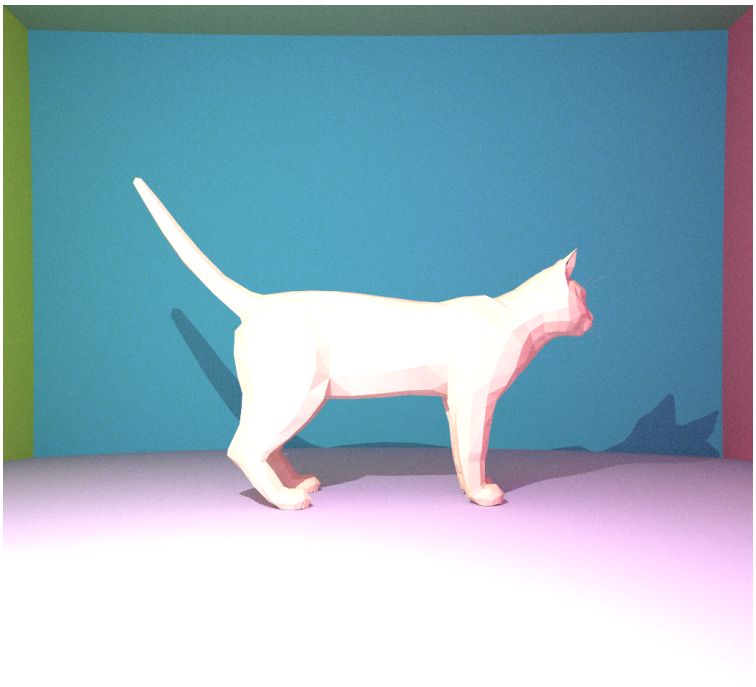


Figure 3: Cat mesh with 3,954 triangles rendered with BVH acceleration, global illumination, and 64 samples per pixel.

| Scene | Resolution | SPP | Triangles | Time |
|---|---|---|---|---|
| Cornell box, spheres only | $1024 \times 1024$ | 64 | – | $\approx 5$ s |
| Mirror sphere scene | $1024 \times 1024$ | 64 | – | $\approx 6$ s |
| Cat mesh with BVH | $1024 \times 1024$ | 64 | 3,954 | 9.1 s |

Table 1: Measured render times on an Apple M3 Pro using OpenMP.

The cat scene remains reasonably fast despite its triangle count because the BVH avoids testing all triangles for every ray. Although this table does not include a full no-BVH timing comparison, the implementation makes clear why the acceleration structure is necessary: without it, every ray reaching the mesh would require a linear scan over all triangles, while the BVH allows most irrelevant triangle groups to be skipped early.

The rendering time is also strongly affected by the number of samples per pixel and the recursion depth. Increasing the sample count reduces Monte Carlo noise and improves the stability of indirect lighting, but it also increases render time almost linearly. Similarly, increasing the maximum recursion depth allows more light bounces, but each additional bounce increases the number of rays traced in the scene.

# 6 Limitations and Conclusion

Several optional features were not implemented in this version: transparent materials, area lights, depth of field, texture mapping, and Phong normal interpolation. The path tracer also uses a fixed maximum recursion depth instead of Russian roulette termination. The BVH uses a simple longest-axis midpoint split, which is easy to implement but less optimal than more advanced strategies such as surface area heuristic splitting.

Overall, I implemented a complete basic path tracer in C++. The renderer supports diffuse and mirror materials, direct lighting, hard shadows, indirect lighting, anti-aliasing, triangle mesh rendering, and BVH acceleration. The visual results show the effect of each feature: direct lighting creates hard black shadows, indirect lighting fills these regions with bounced light, anti-aliasing smooths object edges, and mirror reflection correctly shows the surrounding scene. The final cat render demonstrates that BVH acceleration makes triangle mesh rendering practical by avoiding unnecessary intersection tests.

This project also helped me connect the different parts of a rendering pipeline. The final image depends not only on the shading model, but also on numerical details such as ray offsets, sampling quality, recursion limits, and acceleration structures. Each lab added one layer to the renderer, and the final result shows how these components work together to produce a path-traced image.

## References

Course lecture notes, Computer Graphics, Ecole Polytechnique, 2025–2026.